System Architecture

River Hawk Consulting, LLC · IMAX HCD Prototype OT
August 8, 2026

CONTENTS

System Architecture

Agreement No. 5600002690012 · River Hawk Consulting, LLC · UNCLASSIFIED IMAX Chat-Viewer Prototype

Delivered under **Article XII, Data Rights**: *”the architecture the Performer will deliver prior to the end of the period of performance.”*

Article I(b) defines what that word has to mean here:

> *”The overall design and structure of the system, including the relationships between its components, interfaces, and data flows… sufficient to enable the Government to implement the system without requiring further development or support from the Performer.”*

That last clause sets the bar. This document is written to it: it describes not only how the prototype is put together but everything the Government needs in order to stand it up, change it, and cut it over to production services without asking River Hawk anything. Where something genuinely cannot be known outside the enclave, it says so and names the value that has to be supplied.

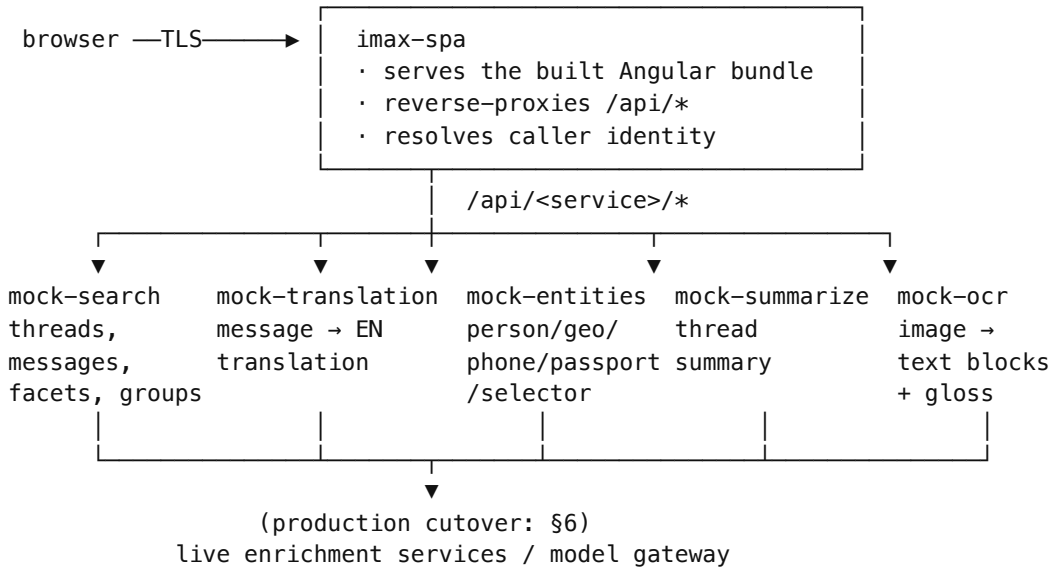
Companion documents: `developer_guide.md` (how to work in the codebase day to day), `deploy/README.md` (deployment procedure), `deploy/AIRGAP.md` (restricted-network transfer). This document is the one that has to stand alone.

---

1. What the system is

A single-page application for linguists and targeting officers triaging multi-language intercepted communications, plus a service layer that emulates the production enrichment services behind frozen contracts.

Six containers. One serves the SPA and proxies the API surface; five are single-responsibility enrichment services. Nothing shares a database, because nothing has one — see §5.



WHY SIX AND NOT ONE

Each service owns one contract and can be replaced independently. At cutover the Government does not swap “the backend” — it repoints one service at a time and keeps the rest on fixtures until each is proven. The seam is per-service by design, and §6 is the procedure.

---

2. Components

2.1 `IMAX-SPA` — SPA HOST AND API EDGE

- **Image**: `deploy/Dockerfile.spa`. Node runtime, non-root, read-only root filesystem.
- **Entrypoint**: `deploy/spa-entry.js`. An Express edge, not nginx — deliberately, so the identity logic and the proxy live in the same reviewable process as the TLS termination.
- **Responsibilities**
  1. Serve the compiled Angular bundle and the fabricated corpus’s static assets.
  2. Reverse-proxy `/api/<service>/*` to the corresponding service, per `UPSTREAM_*`.
  3. Resolve caller identity from what the authenticating front sends, and refuse unauthenticated requests (§4).
  4. Terminate TLS using material fetched at pod start (§3).

2.2 THE FIVE ENRICHMENT SERVICES

All five share `deploy/Dockerfile.mock` and `deploy/mock-entry.js`; the route module differs.

Service	Route module	Contract	Production analogue
`mock-search`	`routes/search.js`	threads, thread messages, message search, facet counts, groups	Corpus search / retrieval
`mock-translation`	`routes/translation.js`	message → English, with service attribution	Machine translation
`mock-entities`	`routes/entities.js`	message → typed entities with confidence	Entity extraction
`mock-summarize`	`routes/summarize.js`	thread → executive summary	Summarization
`mock-ocr`	`routes/ocr.js`	attachment → text blocks, per page, plus English gloss	OCR / document exploitation

Each answers **schema-pinned JSON**. The schemas are the interface contract, not an implementation detail: the client's typed models mirror them exactly, so a drift surfaces as a compile error rather than a runtime surprise. `tests/services.spec.js` asserts the wire format independently of the UI, which is what makes the contracts testable at cutover.

2.3 THE CLIENT

Angular 21.2.19, pinned exactly. Standalone components, signals, no NgModules.

```
angular/src/app/
  app.ts / app.html          shell: toolbar, UNCLASS banner, three panels
  core/
    models/                  typed mirrors of the service contracts
    api/                     one wrapper per service
    stores/                  triage · session · gold-copy (signal state)
    services/identity        who the front says is signed in
    directives/panel-resize  shared drag-to-resize
  features/
    triage/                  Search & Triage — queue, facets, groups, filters
    viewer/                  chat stream, message bubble, OCR scan viewer
    gold-copy/               promoted threads and export
```

Three stores, three lifetimes. `TriageStore` holds what is selected. `SessionStore` holds the officer's work — cached translations, review verdicts, notes, document tags. `GoldCopyStore` holds promoted threads and their order. All three are in-memory signals with no persistence layer, which is a requirement rather than an omission (§5).

---

3. Certificates and TLS

Serving TLS requires a certificate and key. In the target environment these live in AWS Secrets Manager behind a role, so the cluster fetches them itself rather than a human staging a Kubernetes Secret.

Flow. An initContainer (`deploy/secrets-init.js`) runs before the application container, authenticates, fetches the material, validates it, and writes `tls.crt` / `tls.key` / `ca.pem` mode `0600` into an in-memory `emptyDir`. The application container mounts the same volume at `/etc/tls` and reads it synchronously at import (`deploy/tls.js`). The private key never becomes a Kubernetes Secret and never touches disk.

Credential chain, in order, so both credential models work with the same manifests:

1. `AWS_WEB_IDENTITY_TOKEN_FILE` + `AWS_ROLE_ARN` → `AssumeRoleWithWebIdentity` (IRSA)
2. `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` / `AWS_SESSION_TOKEN`
3. IMDSv2 → instance profile

Secret layouts handled without a code change, because the layout was not knowable in advance: two secrets (cert and key separately); one secret carrying both; raw PEM; JSON under any of several key spellings; and base64 inside JSON. Anything else is refused by name, loudly, before the pod starts — a wrong secret fails here rather than three layers down as a TLS handshake error.

Signing is SigV4 over `node:crypto` (`deploy/aws-sigv4.js`), with no AWS SDK. The SDK would have added roughly forty transitive packages that the enclave's npm mirror has to carry and the vulnerability gate has to clear, for one signed POST.

What the Government must supply at deploy time — none of it is in the repository:

Value	Where	Why absent
`AWS_SECRETSMANAGER_ENDPOINT`	`imax-tls-source` ConfigMap	Regional endpoint; committing enclave hostnames is blocked by `scripts/preflight-airgap.sh`
`TLS_CERT_SECRET`, `TLS_KEY_SECRET`, `TLS_CA_SECRET`	same	Secret names are unknown outside the account
Role ARN	`deploy/k8s/serviceaccount.yaml` annotation	Unset falls through to the node role

Overlay: `deploy/overlays/tls-awssm`. The file-based overlay `deploy/overlays/tls` is retained for environments with no AWS.

---

4. Identity and authorization

The prototype does not authenticate anyone. An authenticating front does, and forwards the result; the pod trusts that front, which is the same posture the deployment already takes by having no other route in.

Two modes, selected by `AUTH_MODE`:

- **`bearer-jwt`** (the target). The front sends the identity token in `Authorization` and the access token in `x-auth-request-access-token`. The pod base64url-decodes the payload and reads claims. **The signature is not verified** — deliberately, and logged at startup so the assumption is visible in `kubectl logs` rather than buried in a comment.
- **`proxy-header`**. Flat headers (`x-ain`, `x-name`, `x-ismemberof`, `x-org`) for any front that sends them.

Both produce the same `{id, name, org, groups, label}`, so everything downstream — group authorization, `/api/whoami`, the toolbar — is mode-agnostic.

Claim names are configuration, not code: `AUTH_CLAIM_ID`, `AUTH_CLAIM_NAME`, `AUTH_CLAIM_GROUPS`, `AUTH_CLAIM_ORG`. The defaults (`sub`, `name`, `groups`, `org`) are the common OIDC spellings and are **not confirmed** against the Sponsor's STS. `/api/whoami` reports which claim names a real token actually carries — names only, never values — so a mismatch is diagnosed in one request and corrected with a ConfigMap edit.

`AUTH_REQUIRED_GROUPS` gates access on group membership. Empty means any authenticated caller.

---

5. State, data and persistence

The system stores nothing. No database, no cache, no session store, no client-side persistence — no `localStorage`, `sessionStorage`, `IndexedDB` or cookies anywhere in either build. Officer work — translations, verdicts, notes, tags, promoted threads, panel widths — lives in memory for the life of the tab.

This is a deliberate prototype property with two consequences the Government should weigh:

- **Every pod is disposable and horizontally scalable.** Any replica can serve any request.
- **A reload loses the officer's work.** For a prototype demonstrating a workflow this is correct; for production it is the first thing to specify, and the seam is `SessionStore` — a single injectable holding all of it.

The demonstration corpus (`data/seed.js`) is **fabricated, unclassified** demonstration data: 45 threads across Arabic, Farsi, Chinese and Russian, three ingest lanes (message, transcript, document), and document scans including a five-page Arabic customs declaration. Every screen carries an UNCLASS banner.

---

6. Production cutover

The point of the seam. Per service, not all at once:

1. Point `UPSTREAM_<SERVICE>` at the production service.
2. Confirm the response satisfies the pinned schema — `tests/services.spec.js` is the check, and it runs against whatever the variable points at.
3. Repeat. Services still on fixtures keep working; there is no flag day.

Model gateway. The enrichment services call a live gateway when `MODEL_ENDPOINT`, `MODEL_NAME` and `MODEL_API_KEY` are supplied, and fall back to fixtures otherwise. Three things fail in ways that do not look like their cause, so they are stated here:

- `enabled()` requires **both** endpoint and key. Miss either and the pods answer from fixtures — a green deployment quietly not using the gateway. Each pod logs which mode it is in at startup.
- The endpoint must be the base URL **including the version path** (`https://host/v1`); the code appends `/chat/completions`. A URL with the suffix already on it 404s in a way that reads like an outage.
- The host is parsed into the provenance label an officer sees on every translation, so a malformed endpoint corrupts the gold copy's audit trail, not just the request.

`MODEL_ENDPOINT` and `MODEL_NAME` live in a ConfigMap; only `MODEL_API_KEY` is a Secret. A URL is not a credential, and burying it in a Secret means it cannot be diffed or corrected without recreating the whole object.

---

7. Deployment

Two equivalent descriptions, and CI fails if they diverge:

- **kustomize** — `deploy/k8s/` with overlays for TLS.
- **Helm** — `deploy/chart/`, for ArgoCD.

`scripts/compare-chart-kustomize.py` renders both and compares every behaviour-deciding field. It is a build gate, not a convention: the two cannot drift silently.

Posture. All six pods: non-root, read-only root filesystem, dropped capabilities, `seccompProfile: RuntimeDefault`, no privilege escalation — the Restricted Pod Security Standard. Resource limits are 250m CPU / 256Mi memory each, inside the TDD's 0.5 vCPU / 512 MB ceiling. A NetworkPolicy restricts service traffic to the SPA pod. Probes are scheme-aware, which matters: leaving them on HTTP after enabling TLS produces an infinite restart loop that looks like a crash.

Restricted-network transfer (`deploy/AIRGAP.md`): fonts vendored so the SPA fetches nothing at runtime; base images digest-pinned; Dockerfiles parameterized for in-enclave bases and mirrors; a registry-prefix transform; an optional enclave-CA mount; and an architecture gate that refuses to package images built for the wrong CPU — an arm64 bundle loads into an x86_64 cluster without complaint and then every pod dies with an exec format error.

Document scans are served as **rasters**, not SVG. The SVG sources name Arabic and CJK fonts that exist on macOS and not on a typical Linux workstation; rasterizing removes the enclave's dependency on having them.

---

8. Interfaces

8.1 EXTERNAL

Interface	Direction	Protocol	Notes
Browser → `imax-spa`	in	HTTPS	TLS terminated in-pod (§3)
Authenticating front → `imax-spa`	in	headers on each request	§4
`imax-spa` → enrichment services	out	HTTP, in-cluster	`UPSTREAM_*`
Enrichment services → model gateway	out	HTTPS	Optional; fixtures otherwise (§6)
initContainer → Secrets Manager	out	HTTPS, SigV4	Once at pod start (§3)

8.2 API SURFACE

All under `/api`, all JSON, all schema-pinned:

```
GET  /api/search/threads                        thread list + facet counts
GET  /api/search/threads/:threadId/messages     thread message stream
GET  /api/search/messages                       corpus-wide search
GET  /api/search/groups                         geo-fence and watchlist groups
POST /api/translate                             message → English
POST /api/entities                              message → typed entities
POST /api/summarize                             thread → summary
POST /api/ocr                                   attachment → blocks + gloss
GET  /api/whoami                                resolved caller identity + diagnostics
```

8.3 CONFIGURATION

Everything environment-specific is a ConfigMap or Secret value; nothing enclave-specific is committed, and `scripts/preflight-airgap.sh` fails the build if it ever is.

Object	Carries
`imax-auth` (ConfigMap)	`AUTH_MODE`, header and claim names, `AUTH_REQUIRED_GROUPS`
`imax-model-gateway` (ConfigMap)	`MODEL_ENDPOINT`, `MODEL_NAME`
`imax-model-gateway` (Secret)	`MODEL_API_KEY`
`imax-tls-config` (ConfigMap)	TLS file paths, client-auth mode, upstream scheme
`imax-tls-source` (ConfigMap)	AWS region, Secrets Manager endpoint, secret names
`imax` (ServiceAccount)	IRSA role ARN annotation

---

9. Rotation, scaling and failure

- **Certificate rotation** — the initContainer fetches once at pod start, so a rotated certificate is picked up by `kubectl rollout restart`. Continuous refresh would need External Secrets Operator or a sidecar; noted, not built.
- **Scaling** — every pod is stateless, so replica count is a free variable.
- **Failure modes** and their triage lines are tabulated in `deploy/AIRGAP.md`. The ones worth knowing here: missing certificate material holds the pod in `Init:Error` rather than letting it serve plaintext on a port the cluster believes is TLS; a missing model gateway falls back to fixtures and says so at startup; a claim-name mismatch 401s every request and is diagnosed through `/api/whoami`.

---

10. What the Government still has to supply

Nothing in this list is a defect; each is a value that cannot be known outside the enclave. They are gathered here so that standing the system up is a checklist rather than an investigation.

#	Value	Consequence if wrong
1	Secrets Manager endpoint and whether a VPC endpoint exists in the target VPC	`ENOTFOUND` at pod start; no route to the API
2	Certificate secret names, and the role granting `secretsmanager:GetSecretValue`	`AccessDenied` or a named not-found failure at pod start
3	Whether the certificate's SANs cover all six in-cluster service names	502 after enabling TLS — the likeliest failure
4	The STS claim names a real token carries	401 on every request; `/api/whoami` reports the actual names
5	`MODEL_ENDPOINT`, `MODEL_NAME`, `MODEL_API_KEY`	Silent fixture fallback, visible in the startup log
6	In-enclave base images and npm mirror	Image build fails in the enclave

Items 1–4 are ConfigMap edits. Item 5 is a ConfigMap and a Secret. Item 6 is documented in `deploy/AIRGAP.md` and parameterized in both Dockerfiles. **None requires a code change**, which is the property Article I(b) is asking about.